Rescue Operations System (ARDMS) - Project Architecture & Workflow

This document explains the overarching system architecture, logic loops, and the agentic workflow (for AI assistants modifying this codebase).

1. System Architecture & Core Files

- **`system-backend/server.js`**: The central nervous system. Handles REST API requests, SQLite database operations, and acts as the master WebSocket broadcaster.
- **`preview-mobile-app.html`**: The Public SOS interface. Collects citizen data, compresses media locally, fetches high-accuracy GPS coordinates, and POSTs to `/api/rescue-request`.
- **`preview-rescuer.html`**: The Field Responder interface. Maintains a persistent WebSocket connection, listens for `NEW_COMMAND`, and POSTs live location data every 10 seconds.
- **`raw_admin.html` (Compiled to `Web ADMIN.html`)**: The Headquarters interface. Consumes the WebSocket feed to instantly update maps, tables, and AI configuration settings in real-time.

2. Artificial Intelligence Engine Workflow

The system incorporates a dynamic AI assignment loop inside `server.js` designed to automate emergency dispatching:

1. **Trigger Phase**: The AI Timer (configurable via the Web Admin) polls the `rescue_requests` table for any `status = 'pending'`.
2. **Proximity Phase**: For each pending SOS, the engine queries the `users` table for online rescuers. It calculates the Haversine distance between the rescuer and the SOS location.
3. **Filtering Phase**: It filters out any rescuers currently occupied (e.g., `status = 'busy'`), and ignores rescuers who have already been assigned to this specific SOS (recorded in `sos_assignment_history`).
4. **Assignment Phase**: It dispatches the task to the nearest eligible rescuer via a targeted WebSocket `NEW_COMMAND` event.
5. **Reassignment Loop**: If a rescuer ignores the command for a set duration (Buffer Time), the AI marks them as ignored in the history, reverts the task to `pending`, and the loop restarts, guaranteeing the SOS is dynamically passed down the chain of command until accepted.

3. Data Integrity & Concurrency

- **Database Locks**: `server.js` implements active memory locks (`activeSubmissions`, `activeCommandUpdates`) to intercept duplicate network requests from users frantically mashing submission buttons.
- **Auto-Refresh Protocols**: Configuration changes broadcast `SETTINGS_UPDATED`, immediately triggering silent, under-the-hood API fetches on all connected Admin clients to keep UIs perfectly synchronized without manual page refreshes.

4. Agentic Workflow (For Antigravity & AI Developers)

If you are an AI assistant or new developer modifying this project:

1. **Always read this file first** to understand the system layout.
2. Edit the HTML files (`preview-*.html` or `raw_admin.html`) and the backend logic (`server.js`) directly.
3. **NEVER manually edit `htmlStr.js`**. Always use the Python build scripts (e.g., `python sync_apps.py`) to inject your HTML changes into the React Native shells.
4. Adhere to the established fallback UI logic: Ensure graceful failures for GPS timeouts, WebSocket disconnects, or database lock rejections (e.g., Error 429).